MOBL-01: Mobile Monitoring Fundamentals

> **Series:** MOBL — Mobile Monitoring | **Notebook:** 1 of 12 | **Created:** February 2026 | **Last Updated:** 10/02/2026

Overview

This notebook introduces Dynatrace mobile Real User Monitoring (RUM) -- the architecture, supported platforms, mobile entity types, and how beacon data flows from device to Grail. Whether you're a mobile developer instrumenting an app or an SRE analyzing mobile performance, this is your starting point.

OneAgent for Mobile 8.337 (April 2026): What's New

Sprint 8.337 of OneAgent for Mobile lands platform updates and developer-tooling changes that affect both new instrumentations and existing app builds.

Item	Detail	Impact
Android 17 support	OneAgent Mobile now supports Android 17	Verify build/test targets; no app code changes needed
Gradle 9.5 compatibility	Android Gradle plugin works with Gradle 9.5	Update `gradle-wrapper.properties` if pinned to older versions
Swift Instrumentor: macOS 11.5+ required	Build-time instrumentation tool now needs newer macOS	**Build pipeline impact** — update CI runner images for iOS builds
Session-level RUM toggle (Android)	The New RUM Experience can be toggled at session start without app restart	Faster A/B testing of monitoring config
Touch event stability fix (Android)	Better protection against invalid pointer states during multi-touch	Reduces crashes on gesture-heavy apps

Item	Detail	Impact
I/O blocking ANR fix (Android)	Resolved blocking I/O during startup that triggered ANR	Faster perceived startup; fewer ANR reports
SwiftUI button + Tab/TabSection (iOS)	Fixed instrumentation for button labels and Tab/TabSection components	Re-run instrumentor on apps that previously failed
Modal animation performance (iOS)	Fixed slow/stuttering modal presentation animations	UX improvement

> **Action:** if your CI runs Swift Instrumentor on a macOS image older than 11.5, plan the runner-image upgrade before adopting 8.337. iOS builds otherwise still produce instrumented binaries on the prior agent.

OneAgent for Mobile 8.345 (August 2026)

Item	Detail	Impact
Gradle 9.7 compatibility	The Dynatrace Android Gradle plugin now supports Gradle 9.7	Unblocks Android projects already on 9.7; update `gradle-wrapper.properties` if pinned
Android fixes (build 8.345.1)	ANR eliminated in Jetpack Compose instrumentation with Session Replay, and in Compose user-interaction monitoring; 5G NSA sessions no longer misreported as 4G; native crash attribution corrected across sessions	If Compose + Session Replay was deferred over ANR reports, this is the build that addresses them; network-type analytics shift for 5G NSA users
iOS fixes (build 8.345.1)	User-action navigation and web-request detection improved; Session Replay stack overflow prevented; Initial Loading Action tracking corrected; iOS 27 tab-switch Session Replay fixed; a configuration-handling race condition eliminated	Several affect *measured values*, not just stability — Initial Loading Action in particular, so expect a step in that metric after adoption

Released 08/03/2026 with rollout from 08/11/2026. No Flutter, React Native, or Cordova changes in this sprint. As always with mobile, the agent version that matters is the one **compiled into shipped app builds** — these land as users update, not when the tenant updates.

Table of Contents

1. What is Mobile RUM?
2. Mobile Monitoring Architecture
3. Supported Platforms & SDKs
4. Mobile Entity Types
5. Data Flow: Beacon to Grail
6. Mobile vs Web RUM
7. Getting Started Checklist

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`storage:smartscape:read` (app inventory; classic table: `storage:entities:read`), `storage:user.events:read`, `storage:user.sessions:read`
Mobile App	At least one configured mobile application

1. What is Mobile RUM?

Real User Monitoring (RUM) for mobile captures what real users experience when they interact with your native mobile applications. Unlike synthetic monitoring that simulates users, mobile RUM records actual sessions from real devices in the field.

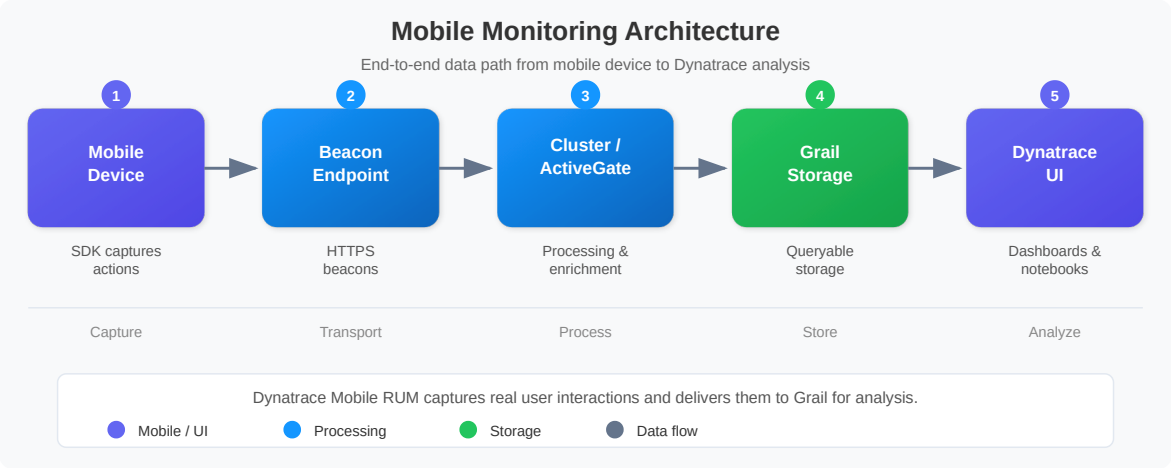
Dynatrace mobile RUM automatically captures:

- **User actions** -- taps, swipes, screen loads, and custom actions
- **Sessions** -- complete user journeys from app launch to background/close
- **Crashes and ANRs** -- native crashes, unhandled exceptions, and Android Application Not Responding events
- **Network requests** -- all HTTP/HTTPS calls made by the app, including response times and errors
- **Device context** -- OS version, device model, carrier, connection type, battery level

Key Mobile Metrics

Metric	Description	Why It Matters
App Launch Time	Time from tap to first interactive screen	Directly impacts user retention
Crash Rate	Percentage of sessions with a crash	Top driver of app store ratings
User Actions per Session	Average number of interactions	Measures engagement depth
Network Error Rate	Percentage of failed HTTP requests	Indicates backend or connectivity issues
Session Duration	Average time users spend in the app	Correlates with business value
Apdex Score	User satisfaction index (0-1)	Single number summarizing performance

2. Mobile Monitoring Architecture



How It Works

1. **SDK embedded in app** -- The Dynatrace mobile SDK is integrated into your app during the build process. It hooks into lifecycle events, network layers, and crash handlers automatically.
2. **Collects telemetry** -- As users interact with the app, the SDK captures user actions (taps, screen loads), network requests (URL, status code, duration), crashes (stack traces, thread state), and device context.
3. **Sends beacons** -- Captured data is packaged into compressed beacon payloads and sent to the Dynatrace beacon endpoint over HTTPS. Beacons are batched and buffered for efficiency.
4. **Processed by cluster** -- The cluster (or ActiveGate) receives beacons, validates them, enriches with server-side correlation data, and normalizes fields.
5. **Stored in Grail** -- Processed mobile data is stored in Grail, making it queryable via DQL alongside logs, spans, metrics, and other telemetry.
6. **Analyzed in UI** -- Use Dynatrace dashboards, notebooks, and the mobile app overview to visualize performance, detect regressions, and drill into individual sessions.

3. Supported Platforms & SDKs

Dynatrace provides native SDKs for all major mobile platforms and cross-platform frameworks:

Platform	SDK	Language	Auto-Instrumentation
iOS	Dynatrace iOS Agent	Swift, Objective-C	UIKit (runtime), SwiftUI (build-time `DTSwiftInstrumentor`)
Android	Dynatrace Android Agent	Kotlin, Java	Activities, Fragments, Jetpack Compose (plugin 8.271+)
Flutter	dynatrace_flutter_plugin	Dart	Navigation, HTTP
React Native	@dynatrace/react-native-plugin	JavaScript/TypeScript	Navigation, Fetch/XHR
Cordova	dynatrace-cordova-plugin	JavaScript	WebView-based
.NET MAUI	Dynatrace.OneAgent.MAUI	C#	Platform views
Xamarin	Dynatrace.OneAgent.Xamarin -- **end of support May 2025**; migrate to .NET MAUI	C#	Platform views

SDK Integration Methods

Method	Description	Best For
Auto-instrumentation	SDK automatically captures lifecycle events, network calls, and crashes	Most applications; fastest setup
Manual instrumentation	Developer explicitly starts/ends actions and reports values	Custom user actions, business events
Hybrid	Auto-instrumentation + manual calls for custom events	Best of both worlds

> **Note:** Auto-instrumentation coverage varies by platform and mechanism. UIKit and Android Views are instrumented at runtime/bytecode level; SwiftUI needs the build-time SwiftUI instrumentor (MOBL-02 §5); Jetpack Compose is auto-instrumented by default from Android Gradle plugin 8.271 (MOBL-03 §5).

4. Mobile Entity Types

Dynatrace models mobile monitoring data using specific entity types and data objects:

Entity Type	DQL Identifier	Description
Mobile Application	Classic: `dt.entity.mobile_application` — Smartscape: `smartscapeNodes "FRONTEND"` filtered on `frontend.type == "mobile"`	The configured mobile app (iOS, Android, or hybrid)
User Action	`user.events` with `characteristics.has_user_action`	Tap, swipe, screen load, or custom action
Session	`user.sessions` (one record per session)	User session container linking all actions in a visit
Network Request	`user.events` with `characteristics.has_request`	HTTP request initiated by the mobile app
Crash	`user.events` with `characteristics.has_crash` (ANR: `characteristics.has_anr`)	Native crash, ANR, or unhandled exception

> **Note:** Mobile user actions, requests, crashes and ANRs are stored as events in the Grail `user.events` store, and sessions in `user.sessions` -- not as traditional entities, and **not** as business events. `dt.rum.application.type == "mobile"` separates mobile from web frontends in both. The mobile application entity represents the configured app itself.
>
> **Correction (09/28/2026).** Earlier revisions of this table said actions, sessions and requests are business events (`bizevents`). That data model does not exist; see MOBL-10 §1 for the mapping and the semantic-dictionary evidence.
>
> **Correction (07/30/2026).** Earlier revisions of this notebook named the mobile application entity `dt.entity.device_application` in prose. **That entity type does not exist.** `verify_dql` reports `The entity type dt.entity.device_application wasn't found` — and then still reports the statement valid, so the query runs and returns zero rows, which is indistinguishable from "this environment has no mobile applications". The only `device_application` types that exist are `dt.entity.device_application_method` and

> `dt.entity.device_application_method_group` (RUM action-level types, not the app). The correct classic type is **`dt.entity.mobile_application`**, which every code cell in this notebook already used — the error was confined to the narrative. Treat a `wasn't found` warning from `verify_dql` as a blocking failure, not advice.

Querying Mobile App Entities

Two surfaces answer this question, and **Smartscape is now the preferred one**. SaaS 1.344 (released 07/27/2026, with a **staged tenant rollout from 07/29/2026**) migrated the Digital Experience apps — Experience Vitals, Users & Sessions, Error Inspector, Synthetic — so that their queries and filters "primarily use `dt.smartscape.*` entities, instead of `dt.entity.*` and `dt.rum.application.*`". Verify 1.344 has reached your tenant before relying on the UI change; the `FRONTEND` node type itself is queryable via DQL independently of it.

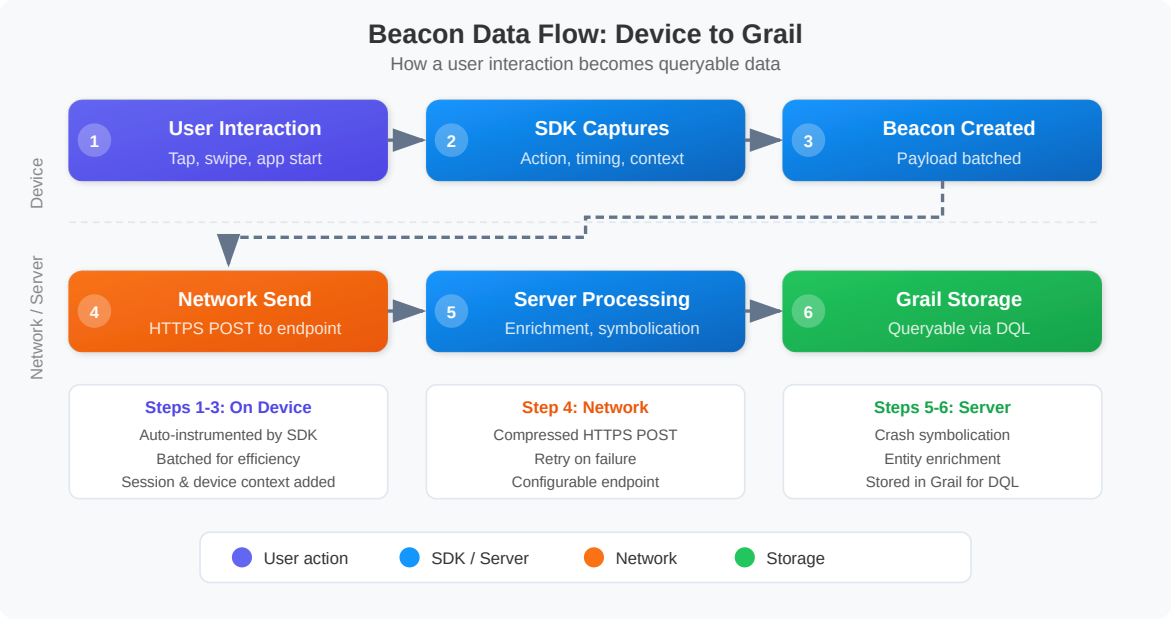
The classic `fetch dt.entity.mobile_application` table **still exists and still returns rows** — unlike some deprecated entity types, this one has not gone hollow. It remains a genuine fallback for tenants awaiting 1.344 and for saved queries still in circulation.

The following query lists all configured mobile applications in your environment:

```
// List all configured mobile applications
// PREFERRED — Smartscape. Mobile and web applications share the FRONTEND node
type;
// frontend.type is the discriminator, so the filter is mandatory for a mobile-
only list.
// id_classic carries the MOBILE_APPLICATION-* id, which joins back to
unmigrated queries.
smartscapeNodes "FRONTEND"
| filter frontend.type == "mobile"
| fields name, id, id_classic, tags
| sort name asc

// FALLBACK (classic surface — still functional, keep while SaaS 1.344 rolls
out):
// fetch dt.entity.mobile_application
// | fields entity.name, id, tags
// | sort entity.name asc
```

5. Data Flow: Beacon to Grail



Beacon Lifecycle Details

Batching: The SDK does not send a beacon for every individual action. Instead, it batches multiple events into a single beacon payload to reduce network overhead and battery consumption.

Offline Buffering: When the device has no network connectivity, the SDK buffers monitoring data locally and sends it once connectivity is restored. The buffer is bounded by the agent, so a long offline period can still lose data — treat the buffer as protection against short gaps (a subway ride, a dead zone), not as a guarantee.

Compression: Beacon payloads are compressed before transmission to minimize bandwidth usage, which is especially important for users on metered cellular connections.

Session Correlation: Each beacon includes a session ID that links all actions from a single user visit. The cluster uses this to reconstruct the full session timeline.

Counting Mobile Applications by Name

Use the following query to see a summary of your configured mobile applications:

```
// Count mobile applications by name
// PREFERRED — Smartscape. Grouping by name preserves the per-app breakdown;
drop the
// filter and group by frontend.type instead to see the mobile-vs-web split in
one pass.
smartscapeNodes "FRONTEND"
| filter frontend.type == "mobile"
| summarize app_count = count(), by:{name}
| sort app_count desc

// FALLBACK (classic surface — still functional):
// fetch dt.entity.mobile_application
// | summarize app_count = count(), by:{entity.name}
// | sort app_count desc
```

6. Mobile vs Web RUM

> **Mobile sessions gain bounce classification (SaaS 1.346 — staged rollout from 08/25/2026).** Mobile sessions are now classified as bounces using the **same definition already applied to web sessions**, which removes one of the mobile-vs-web asymmetries this section documents: bounce rate becomes comparable across both surfaces rather than web-only. It carries a hard floor — **Mobile agent 8.343+** — so an app on an older agent will not produce the classification regardless of tenant version. Check your app's shipped agent version, not just the tenant, before reporting on it.
>
> Source: What's new in Dynatrace SaaS 1.346 (DT docs)

Understanding the differences between mobile and web RUM helps you choose the right approach and set correct expectations for each platform:

Aspect	Mobile RUM	Web RUM
Entity type	Classic `dt.entity.mobile_application` → Smartscape `FRONTEND` with `frontend.type == "mobile"`	Classic `dt.entity.application` → Smartscape `FRONTEND` with `frontend.type == "web"`
Instrumentation	Native SDK embedded in app binary	JavaScript tag injected into page

Aspect	Mobile RUM	Web RUM
Crash detection	Native crash, ANR, unhandled exception	JavaScript errors, promise rejections
Network monitoring	All HTTP/HTTPS calls from the app	XHR, Fetch API calls
Session replay	Screen recording (visual replay)	DOM replay (HTML reconstruction)
Offline capability	Beacon buffering (data saved locally)	No (requires active connectivity)
Distribution	App stores (Apple App Store, Google Play)	Web deployment (CDN, server)
Update cycle	Users must update the app	Instant (server-side deployment)
Device diversity	Thousands of device/OS combinations	Browser/OS combinations
Performance factors	CPU, memory, battery, network type	Browser rendering, network latency

The Entity Model Converged (SaaS 1.344)

On the classic surface mobile and web applications were two different entity types. On Smartscape they are **one node type**: `FRONTEND`. The distinguishing field is `frontend.type` (`"mobile"` / `"web"`) — not the node type. There is **no** **`dt.smartscape.mobile_application`** and no `dt.smartscape.application`; both classic types map onto the single model `dt.smartscape.frontend`, titled *Digital Experience Frontend*.

SaaS 1.344 (released 07/27/2026 — **staged tenant rollout from 07/29/2026**) is what promotes this to the primary surface: the Digital Experience apps moved their queries and filters onto `dt.smartscape.*` entities. Verify 1.344 has reached your tenant before depending on that; the node type is already queryable regardless, and the classic entity tables keep working either way.

What this changes in practice:

- **One inventory query covers both platforms.** `smartscapeNodes "FRONTEND" | summarize apps = count(), by:{frontend.type}` returns the mobile/web split in a single pass — no union of two entity tables.

- **Filtering on `frontend.type` is mandatory in mobile-only queries.** Omit it and a "list my mobile apps" query silently includes every web application as well. There is no error to warn you.
- **`frontend.type` is not listed in the model's own `fields` array**, yet it exists and filters correctly. The Semantic Dictionary field list is an index, not a completeness guarantee — do not conclude a field is missing because it isn't enumerated.
- **`id_classic` is the migration bridge.** Every `FRONTEND` node carries its classic id (`MOBILE_APPLICATION-*` or `APPLICATION-*`), so half-migrated query sets can be joined during the transition.

Key Implications

- **Version fragmentation:** Mobile apps have multiple versions in the wild simultaneously. Your DQL queries should account for `app.version` when analyzing performance.
- **Offline data gaps:** Data from a device that was offline arrives late, when it reconnects. Recent time windows can still fill in after you first query them — re-check before treating a dip in the last hour as real.
- **Platform-specific issues:** iOS and Android have different crash signatures, lifecycle events, and performance characteristics. Analyze them separately when troubleshooting.

7. Getting Started Checklist

Follow these steps to set up mobile monitoring for your application:

1. **Create a mobile frontend and turn on the New RUM Experience** -- Turn on **Enable RUM** for mobile (**Settings > Collect and capture > Real User Monitoring > Enablement and cost control > Mobile**), create the frontend with **Experience Vitals > Add Frontend**, then turn on **New Real User Monitoring Experience** in the frontend's **Settings > Enablement and cost control**. The Grail `user.events` / `user.sessions` queries throughout this series read the data the New RUM Experience sends; with it off they return nothing. MOBL-02 / MOBL-03 have the step-by-step.
2. **Integrate the SDK** -- Add the Dynatrace mobile SDK to your app's build system (CocoaPods/SPM for iOS, Gradle for Android, npm for React Native/Flutter). Follow the platform-specific setup guide.
3. **Configure the app ID and beacon URL** -- Set the application ID and beacon endpoint URL provided by Dynatrace in your SDK configuration file.

4. **Build and test** -- Build the app with the SDK integrated, launch it on a test device, and verify that sessions appear in the Dynatrace UI within a few minutes.
5. **Verify data in Grail** -- Use DQL to confirm that mobile telemetry is flowing into Grail. Run the entity query below to check your app appears.
6. **Configure user action naming rules** -- Customize how user actions are named in Dynatrace to make them meaningful for your team (e.g., "Tap on Checkout" instead of generic "Touch on Button").
7. **Set up crash symbolication** -- Upload dSYM files (iOS) or ProGuard/R8 mapping files (Android) so that crash stack traces show human-readable class and method names.
8. **Enable session replay (optional)** -- Turn on mobile session replay to capture visual recordings of user sessions for debugging UI issues.
9. **Define custom actions (optional)** -- Use the SDK API to report custom user actions and business events that are specific to your app's workflow.
10. **Create dashboards and alerts** -- Build dashboards for crash rate, app launch time, and network errors. For alerting, use **Davis anomaly detectors** rather than classic alerting profiles: `builtin:alerting.profile` is flagged blocked at upgrade, so a checklist that ends there leaves you building something with an expiry. See ALERT-01 for the target architecture and MOBL-11 §5 for a mobile crash-rate detector.

Verify Your Mobile App Configuration

Run this query to confirm your mobile applications are configured and visible in Dynatrace:

```
// Mobile app inventory overview
// PREFERRED — Smartscape
smartscapeNodes "FRONTEND"
| filter frontend.type == "mobile"
| fields name, id, id_classic, lifetime, tags
| sort name asc
| limit 20

// FALLBACK (classic surface — still functional):
// fetch dt.entity.mobile_application
// | fields entity.name, id, lifetime, tags
// | sort entity.name asc
// | limit 20
```

Summary

In this notebook, you learned:

- **What mobile RUM is** and the key metrics it captures (app launch time, crash rate, user actions per session, network error rate)
- **Mobile monitoring architecture** -- how the SDK, beacon endpoint, cluster, and Grail work together
- **Supported platforms** -- iOS, Android, Flutter, React Native, Cordova, and .NET MAUI (Xamarin is past end of support)
- **Mobile entity types** -- the `FRONTEND` Smartscape node filtered on `frontend.type == "mobile"` (preferred), or classic `dt.entity.mobile_application` (still functional) -- **never** `dt.entity.device_application`, which does not exist -- plus `user.events` / `user.sessions` for actions, sessions, requests, and crashes
- **Why mobile and web share one node type** -- Smartscape collapsed both classic application entities into `FRONTEND`, discriminated by `frontend.type`, with `id_classic` bridging back to classic ids
- **Beacon data flow** -- batching, offline buffering, compression, and session correlation
- **Mobile vs Web RUM differences** -- entity types, instrumentation methods, offline capability, and platform-specific considerations
- **Getting started steps** -- from SDK integration through dashboard creation

---

Next Steps

Continue to **MOBL-02: iOS SDK Setup (Swift & SwiftUI)** to learn:
- Creating a mobile app configuration and installing the iOS SDK (SPM or CocoaPods)
- Configuring `Info.plist` for auto-start and crash reporting
- UIKit auto-instrumentation and SwiftUI instrumentation with `DTSwiftInstrumentor`
- Verifying data flow with DQL

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.